

As the Scrum Master of SNHU Travel development pilot, I conducted the Sprint Review and Retrospective to assess the feasibility of implementing a Scrum-Agile methodology in ChadaTech. I changed roles during development several times over the weeks of the project, in order to gain a better understanding of what each role does to help ensure that our product will inevitably be a success. This retrospective looks at the extent that Scrum roles supported with development, how user stories moved from inception to completion, how interruptions were dealt with, and finally a look at how communication tools and organization serve as a foundation for collaboration. It ends with an analysis of whether Scrum-Agile was the best fit for this project.

The defined roles under Scrum-Agile played a big part in being able to achieve team and hence project success. As Product Owner, I converted stakeholder feedback from the focus group into mapped user stories. For example, user needs led to the development of a feature called “Top Five Destination List” as per his input. By moving this feature up in the product backlog, the team was able to deliver something valuable to the customer early.

As a Scrum Master, I led sprint planning and retrospectives and made sure that the team was tracking towards sprint objectives. This position focused on obstacle-removal and on keeping the others on task. So, when the ListView interface had some fuzzy implementation details, Scrum Master served to clarify acceptance criteria rather than being stuck with assumptions.

During my time working as a Development Team member, I concentrated on incrementally adding features such as the ListView control, image inclusion and link support. The self-organizing capabilities facilitated finding workarounds for technical problems, like

rescaling images and including resources properly in a runnable JAR file. Every responsibility reinforced the element of accountability, collaboration, and ownership.

This separation of duties avoided confusion and enabled the class to continue efficiently. With Scrum there were no overlapping roles, instead of stepping on each other's toes it promoted organized teamwork and mutual responsibility.

Scrum-Agile contributed substantially to completing the user stories. User Stories were developed in the form of: "As a user, I'd like to...so that...". What's more, it anchored every task explicitly to the user need rather than just some abstract function. For instance, the user story that wanted to list five destinations with descriptions, an image and embedded links was broken down into smaller implementation tasks of the sprint. Instead of building it all, the team added just a list structure at first, then descriptions, then images, then link behavior and finally visual customizations. "Step by step" means that the feature was developed in small stages, and it worked at every stage of development.

Sprint Reviews served as checkpoints to ensure the implemented solution satisfied acceptance criteria. As the work was iterative, changes could be implemented rapidly. For instance, when we need to fix link behavior, so it worked nicely within the structure of a JAR file, this was sorted out within the sprint, instead of being one in a series of large downstream fixes. The Agile approach to working software over comprehensive documentation guaranteed something tangible at the end of each sprint. It bolstered team confidence and tangibly demonstrated success.

When obstacles arose, one of the advantages of Scrum-Agile became more apparent. During the development, we had to make some adjustments like optimizing image resources, fix link mismatches and repack projects so they could be JAR exported properly. Depending upon

what type of development model the adjustments were being made under, those changes could have taken a formal change request and reapproval of the added phase. Rather, the Scrum framework permitted adjustment throughout sprint cycles. As a result of prioritizing the backlog, new features were added without impacting the overall project's schedule. The short distance style of the sprints meant you could be agile without losing accountability.

Additionally, when reflections were part of retrospectives, improvement could be continual. For instance, heightened awareness about the potential for project packaging and resource management to cause submission problems encourage earlier verification in later stages. The ability to inspect and adapt prevented small problems from turning into bigger or potentially unresolvable problems.

Communication was key to the success of the project. Techniques of communication were user stories, sprint planning points, backlog prioritization, and retrospective recommendations. Every component had a function to preserve it in clarity. For example, the acceptance criteria of user stories was a quantifiable definition of done. That removed ambiguity during development and testing. Furthermore, feedback in the sprint review discussions allowed for any misunderstandings to be addressed and feature refinement. The Daily Scrum format fostered transparency, even when simulated. It was important to see what had been done, what was being worked on and where the blockages were in order to keep motivation and accountability high. Communication was short and with-a-purpose, to avoid wasted time. These were communication techniques that facilitated collaboration and were used to get the team on the same page. Instead of siloing themselves, each position contributed towards specific sprint goals.

The project success also benefited from several Scrum events and tools. The Product Backlog was a central organizing piece enabling straightforward feature prioritization. In Sprint Planning the backlog items are then worked out so that they become bite-sized sprint goals. The Sprint Review was validating if we'd built what was useful and the most current version of it, while the Sprint Retrospective focused on learning and improving. Transparency was enhanced through visual order of things, such as user-stories, and criteria for acceptance. The principles of inspection and adaptation were particularly useful. The iterative approach also minimized risk by enabling incremental enhancement and not having the whole thing built before it was tested.

By breaking development down into bite-sized sprints, the team was able to maintain a steady pace and limit the uncertainty. This architecture was particularly helpful for technical requirements as image resource handling and runnable JAR considering.

The Scrum-Agile methodology had advantages and disadvantages in the implementation of SNHU Travel.

Pros:

- Increased adaptability when requirements evolved
- Clear prioritization through the backlog
- Improved stakeholder alignment
- Continuous feedback through sprint reviews
- Enhanced collaboration across roles

Cons:

- Discipline is needed if in Scrum practices are not maintained.
- Demands consistent communication
- Feels less predictable than waterfall at early stages of development
- Lacks cultural fit with traditional models

Scrum-Agile was the appropriate approach in developing SNHU Travel. The project was one with changing specs and successive optimization rounds, especially when it came to

translating user feedback into trackable UI pieces. The adaptability of Scrum enabled the team to pivot fast and create without any sacrifice in quality. If the project had been heavily regulated or it was a rigid documentation with fixed requirements then maybe a waterfall would have been more appropriate. But, for new and customer focused applications, Agile was more responsive and collaborative.

The Scrum-Agile model has been proven through the initial run of SNHU Travel pilot to improve flexibility, collaboration and stakeholder involvement. Clear roles helped with accountability. Iterative software development made it possible for user stories to get finished and structured communication kept the team in good shape. Adaptation from the long-time waterfall paradigm and disciplined facilitation are necessary for there is potential gain in carrying forward the benefits we observed, with this pilot at ChadaTech, to its all-development teams. Through this Sprint Review and Retrospective, it is evident that the Agile methodology not only improved product delivery but also strengthened team dynamics and organizational learning.